

Accessibility Technology Stack

05332/05632/17332/17732 Fall 2026

Introduction to Accessibility and Assistive Technology

<https://cmu-intro-to-accessibility.github.io>

Andrew Begel

Assistive Technologies and Computers

- When developing apps or websites, the typical developer never thinks about accessibility.
- Accessibility itself was not part of computers until 1986.
 - Jim Thatcher invented the screen reader. Ran on IBM DOS.
 - Mac System 4.1 added MouseKeys and StickyKeys in 1987.
- Assistive technologies began as add-on software written by third parties.

Learning Goal

- Familiarity with Assistive Technology (AT)
- Learn about how individual ATs work, who uses them, and their strengths and weaknesses.
- Gain experience with a screen reader and one other AT of your choice.
 - One example: Apple iPhone's [Switch Control](#)
- You will gain practical experience in HW4.

Screen Readers

- An AT that converts on-screen elements into speech or refreshable braille.
- User navigates their computer software using keyboard shortcuts, regular keystrokes, or sometimes touch screen gestures.
- Two modes:
 - Browse: Reading application content, such as docs or webpages.
 - Focus: Used to enter form content.

Screen Readers for Visual Impairments



JAWS



VoiceOver



NVDA



Talkback

Let's use a screen reader!

Practice three core interaction patterns

- Mac/Windows
 1. Linear keyboard navigation through like objects
 2. Hierarchical keyboard navigation through logically-related objects
 3. Keyboard switching between object types
- Phone/tablet
 1. Swipe to navigate linearly
 2. Touch to navigate spatially
 3. First “hit” on an interface element will focus. Double tap to select/activate that interface element.

Now let's zoom!



Now let's listen to a
paper

Accessibility Stack

How might access impact user interfaces?

- Web browsing involves
 - Color
 - Text (shape and size)
 - Content (images, words, sounds, animation)
 - Typing (e.g. input to forms)
 - Moving the mouse (e.g. clicking on navigation links)
 - Comprehension (e.g. reading level)

Differences in abilities all affect access

- Visual impairments
- Differences in understanding content
- Differences in hearing
- Differences in manipulation of mouse or keyboard
- And more.

Goal of Accessibility

- All users should interact with the same website or app as everyone else.
 - Can't force developers to make custom apps for every combination of impairments.
- Users will use browser features or a specialized assistive technology tool to engage with the user interface.
- Thus, the website or app designer must provide the right structure and information to enable assistive technologies to function.

How are we doing?

- 56% of image-based elements are missing labels in Android apps (Fok et al. CHI 2022)
- 82% of Android checkboxes and 62% of Android radio buttons are too small (Ross et al. TACCESS 2020)
- 95%+ Android apps violate accessibility guidelines (Yan and Ramachandran, TACCESS 2019). Many UI elements cannot be focusable, rendering them impossible to use by screen readers.
- Way back in 2019, less than 1% of images on Twitter had ALT-Text. (Gleason et al. WWW 2019)

All digital content must be accessible

- Title II of the Americans with Disabilities Act (ADA) requires governments to ensure that all digital content is fully accessible to people with disabilities.
- That means every academic website at CMU.
- The compliance deadline has been pushed back year and year. Now it's April 26, 2027.

We have to move the needle

- Progress has been slow.
- Shinohara et al. found that less than 50% of US universities taught accessibility at all (SIGCSE 2018).
- You are learning about accessibility here at CMU!

Web Content Accessibility Guidelines (WCAG)

- The Web Accessibility Initiative (WAI), part of the W3C wrote WCAG.
- It makes accessibility recommendations for web authors, browsers, and servers.
 - Focused on accessibility outcomes.
 - Does not tell developers how to achieve those outcomes.

W3C Accessibility Principles - POUR

- **P**erceivable
- **O**perable
- **U**nderstandable
- **R**obust

Perceivable

- Provide **text alternatives** for non-text content.
- Provide **captions and other alternatives** for multimedia.
- Create content that can be **presented in different ways**, including by assistive technologies, without losing meaning.
- Make it easier for users to **see and hear content**.

Operable

- Make all functionality available from a **keyboard**.
- Give users **enough time** to read and use content.
- Do not use content that causes **seizures** or physical reactions.
- Help users **navigate and find content**.
- Make it easier to use **inputs other than keyboard**.

Understandable

- Make text **readable and understandable**.
- Make content appear and operate in **predictable** ways.
- Help users **avoid and correct mistakes**.

Robust

- Maximize **compatibility** with current and future user tools.

How do you conform to WCAG?

- Success criteria: A (lowest), AA, AAA (highest)
- Check them all out here!
 - <https://www.w3.org/WAI/WCAG22/quickref/?versions=2.2&levels=aaa>
- Question? Must every site conform to AAA WCAG?
- Nope. Most apps and websites only meet part of the (A) level compliance.

Why is it so hard to comply?

- When you hear the word “button” in the context of a GUI, what do you think of?
 - Content
 - Structure
 - Style
 - Behavior

Buttons: Under the Hood

- Button doesn't talk directly to screen reader.
- The button provides information to the UI toolkit.
- The UI toolkit talks to the browser which talks to the screen reader.
 - Type (input, button) (Content)
 - Name (Content)
 - Where to put it (Layout)
 - How it looks (Style)
 - How it should react to user input and from which device (Behavior)

Browsers are pretty complex

1. Fetch a webpage
2. Parse the HTML, CSS, Javascript, etc.
3. Build an internal representation of the web page.

Parsing a web page

1. Read the first line fetched from the Internet:

`<!DOCTYPE html>`

- This is a web page!
- Go through the HTML.
 - If you see a tag for a CSS file, load and parse that.
 - If you see a tag for a JS file, load and parse that.
- Render the page.

Getting accessibility information from author to user

- Developers provide accessibility information about the frontend UI to the browser.
 - Or use a UI toolkit library that does.
- Screen readers access the accessibility information through a browser-supplied API.
 - It's like the HTML DOM, but ... different.
- Screen readers output information through audio or Braille.

WAI-ARIA

- WAI-ARIA, the Accessible Rich Internet Applications Suite, ... helps screen readers handle dynamic content and advanced user interface controls developed with HTML, JavaScript, and related technologies.
 - From [WAI-ARIA Overview | Web Accessibility Initiative \(WAI\) | W3C](#)
- Roles
 - Document roles: banner, contentinfo, form, main, navigation, search
 - Widget roles: alert, button, checkbox, link, menuitem, tab, tabpanel
- Properties
 - Drag and drop, Live Region, Relationship, Widget

The first rule of ARIA: Don't write ARIA.

Use native HTML at all times.

Use web frameworks that already have ARIA support.

UI Toolkits all share similar libraries

Java Apps	Web Apps
Java	HTML / CSS / JS
Layouts	CSS Flexbox or Grid
Interactor Hierarchy	Document Object Model (DOM)
Components	<div>
Drawing on the canvas	CSS
<Window onCreate="load">	[window.addEventListener("load", init);]
<Button onAction="#click">	[domElement.addEventListener("click", callback);]

Key Takeaway

As a designer/developer, *YOU* decide who is disabled w.r.t. the technology you create